Wine Quality Prediction with Ensemble Trees: A Unified, Leak-Free Comparative Study

Zilang Chen
South China University of Technology
Guangzhou, Guangdong, China
zilang.chen@outlook.com

Abstract

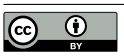
Accurate and repeatable wine quality assessment remains indispensable, but to a large extent, it relies on subjective tasting teams. This study compares five advanced ensemble models—Random Forest, Gradient Boosting, XGBoost, LightGBM, and CatBoost—using the standard Vinho Verde red wine and white wine datasets. Adopt leak-free pipes stratified 80:20 training-test segmentation, five stratified GroupKFolds of the training data, each StandardScaler scaling, SMOTE-Tomek resampling, inverse-frequency cost weighting, Optuna-driven hyperparameter search (120–200 trials per model), and two-stage feature selection modification. Performance is reported on untouched test set, with weighted F_1 as the primary metric. Gradient Boosting provides the strongest test scores weighted $F_1 = 0.693 \pm 0.028$ (red) and 0.664 ± 0.016 (white)—followed closely by Random Forest and XGBoost within 3% points. The input space was compressed to the five most important variables of each model, reducing the dimension by 55%. At the same time, the weight F_1 of red wine was reduced by 2.6% points, and the wright F1 of white wine was reduced by 3.0% points. This confirmed that alcohol, volatile acidity, sulphates, free SO_2, and chlorides occupied the majority of the predictive signal. Runtime analysis on the AMD EPYC 9K84/NVIDIA H_{20} node revealed a distinct efficiency gradient: Gradient Boosting averages 12 hours per five studies, XGBoost and LightGBM complete within 2-3 hours, CatBoost within approximately 1hour, and Random Forest within 50 minutes. These findings indicate that Random Forest is the most cost-effective production model, XGBoost and LightGBM as GPU-efficient alternatives, and Gradient Boosting as the upper limit of accuracy for offline benchmark tests. A fully documented pipeline and comprehensive metric set establish a repeatable baseline for future work on imbalanced, and multi-grade wine quality prediction.

CCS Concepts

• **Computing methodologies** → Machine learning; Machine learning approaches; Classification and regression trees; • **Applied computing** → Computers in other domains; Agriculture.

Keywords

Wine quality prediction, XGBoost, Random Forest, Gradient Boosting, Catboost, LightGBM



IPMLP 2025, Kunming, China

ACM ISBN 979-8-4007-1588-4/2025/07
https://doi.org/10.1145/3759928.3759955

ACM Reference Format:
Zilang Chen. 2025. Wine Quality Prediction with Ensemble Trees: A Unified, Leak-Free Comparative Study. In *The 2nd International Conference on Image Processing, Machine Learning, and Pattern Recognition (IPMLP 2025), July 12, 13, 2025, Kunming, China*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3759928.3759955

1 INTRODUCTION

The assessment of wine quality has mainly relied on subjective judgment of professional sommeliers. However, flavor, aroma, and taste acutally all reflect quantifiable chemical properties, such as acidity, residual sugar, alcohol content and sulphate, etc. Machine-learning models trained based on these physical and chemical indicators have become a scalable alternative. Among them, the Vinho Verde Wine-Quality dataset publiished by Cortez et al. has become the de-facto standard for algorithm testing [1]. Meanwhile, economic research has also revealed the correlation between sensory scores and prices. Budnyak's large-scale exploratory analysis, as well as Palmer and Chen's regression studies, all indicate that higher sensory scores typically correspond to higher prices, although this relationship often exhibits market-specific nonlinear characteristics [2–4]. Despite the continuous emergence of related studies, there is no definite conclusion in the academic community on which prediction method is the best. Liu's research indicates that Support Vector Machine (SVM) performs better in predicting the quality of red wine, whereas Patkar & Chakkaravarthy cite Random Forest and XGBoost have more advantages [5, 6]. After Zaza et al. Corrected the category imbalance, the effectiveness of SVM was once again verified [7]. Rani et al. Further demonstrate that oversampling has an improving effect on multiple models, but XGBoost and Random Forest perform particularly outstandingly [8]. Dahal et al. regarded Gradient Boosting as the best method [9]. The inconsistency of these results[10–17] may stem from the differences in data preprocessing, sample balancing methods and hyperparameter tuning, thus highlightiing the necessity of conducting systematic and comparable research.

In this paper, a systematic and information-leakage-free unified comparison was conducted on five advanced ensemble models (Random Forest, Gradient Boosting, XGBoost, LightGBM, and CatBoost) on the subsets of red and white wine. The main contributions are: (1) Rigorous pipeline: A strict 80/20 stratified training–test split is adopted. A 5-fold Stratified GroupKFold is used within the training set. All preprocessing steps are completed within each training fold. This effectively avoids information leakage and yields unbiased generalization estimates. (2) Systematic hyperparameter search: By using Optuna's Tree-structured Parzen Estimator with median pruning, balanced trial budgets (120–200 times) and consistent early stop

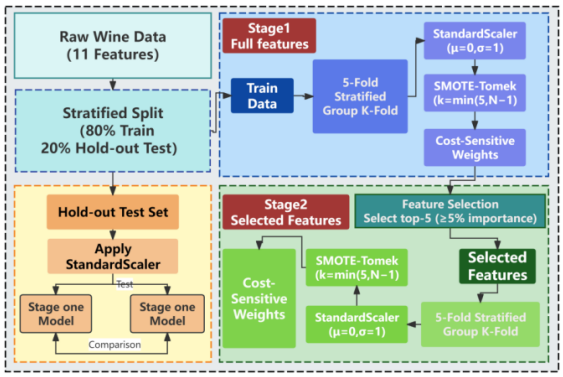


Figure 1: Overview of the Data Preprocessing Pipeline.

rules are allocated to each model to ensure fair optimization depth. (3) Comprehensive evaluation: On the untouched test set, multiple metrics like weighted F_1 were used for evaluation. Meanwhile, the computing resource occupation on the fixed AMD EPYC 9K84 / NVIDIA H20 node was reported, and feature selection was driven by two-stage importance. Quantify the cost–accuracy trade-off. (4) Actionable insights: By providing transparent full-process data and multi-dimensional measurement results, this study establish reproducible benchmarks and decision-making frameworks, providing a basis for selecting ensemble methods in quality-driven, imbalanced, multi-class settings.

2 METHODOLOGY

2.1 Data Sources Introduction and Preprocessing

This paper uses Wine Quality datasets in the UCI Machine Learning Repository, including 1,599 red wine samples and 4,898 white wine samples from the Vinho Verde region of Portugal. Each instance is described by 11 physicochemical features (e.g., acidity, sugar, sulfur dioxide, pH, alcohol) and a quality rating ranging from 0 (very poor) to 10 (excellent). Notably, the distribution of quality labels is highly uneven and basically orderly: most wines are rated as mid-range (quality 5–6), while a very small number are at the extremes.

As shown in Figure 1, Data segmentation, normalization, imbalance mitigation and feature selection were carried out in a fixed sequence. Each dataset was first divided into 80% training data and 20% retained test data through stratified sampling, thereby preserving the original analogy. All subsequent preprocessing, five-fold cross-validation, and hyperparameter tuning are only carried out in the training section, while the testing section remained untouched until the final evaluation. Then, StratifiedGroupKFold is used to segment the training data. This process retains both the label ratio and the natural sample grouping. In each training fold, we applied StandardScaler, use SMOTE-Tomek to balance classes, and trained cost-sensitive models. The corresponding validation folding and retention test sets were scaled using the previously fitted transformer and remain with their native class distributions to ensure unbiased performance estimates. Finally, a two-stage feature selection scheme was adopted to rank the 11 input variables by importance. and the model were retrained on the topmost subset to measure the dimensionality reduction effect. The following details the component of this pipeline:

Group Definition and 5-Fold Stratified Group Split: To prevent information leakage, we assigned a unique group identifier to each wine record and adopted hierarchical group division. After initially stratifying 80% of the training data and 20% of the retained test data, we applied five-fold StratifiedGroupKFold to the training part. This method maintains the class ratio between different folds and stores all samples of the same group in a single fold. Approximately one fifth of the training groups served as validators in each fold, reflecting the overall label distribution. This program prevents nearly duplicate observations from occurring in the training and validation sets, ensuring a strictly leak-free assessment. All hyperparameter optimizations and performance measurements were executed within this cross-validation framework. Training-Fold Processing: In each training fold, all variables were standardized to zero mean and unit variance using the StandardScaler, which is only applicable to training data. This step coordinates the different magnitudes of acidity, sugar, alcohol, and other physicochemical measurements, preventing a wide range of features from dominating the learning process. The fitted scaler was stored and applied to the corresponding validation folds, thereby eliminating leaks. During SMOTE-Tomek, consistent scaling across folds also stabilized distance calculations and improved the numerical tuning of gradient-based models.

To address the severe class imbalance, we adopted the SMOTE-Tomek strategy, combining synthetic oversampling with boundary refinement. SMOTE generates new samples of minority classes through linear interpolation among existing instances in the feature space. The new samples are created as follows:

$$X_{new} = X_i + \lambda \times \left(X_{neighbour} - X_i\right) \quad (1)$$

In the formula, X_i is a minority class sample, $X_{neighbour}$ is one of its k-nearest neighbors, and λ is a random number between 0 and 1. This process increases the density of sparse classes without repeating samples. After oversampling, Tomek link removal is applied to eliminate the majority class samples that form nearest neighbor pairs with the minority class samples. These links are usually lovated near class boundaries and are prone to generating noise or ambiguity. By removing these majority class instances, this method sharpened the decision boundaries and reduced the overlapping class region. This hybrid resampling was performed independently in each training fold. We configured SMOTE with k = min (5, N−1) to ensure that the nearest neighbor oversampling has at least one neighbor even for very rare classes. All experiments used fixed random seeds to ensure reproducibility. Validation and test sets were excluded from resampling, and their original class distributions were retained to provide unbiased performance estimates.

Among all the models, the average pre-resampled imbalance ratio IR_0_before for the red wine was 20.7. and for white wine folding it was 82.9. After treatment with SMOTE-Tomek, the corresponding IR_1_after values decreased to 1.04 and 1.02, and the average IR_improvement iincreased by nearly 20 times and 80 times, respectively. Figure 2 shows that previously, in red wine and white wine, the quality raings of a few categories were respectively below

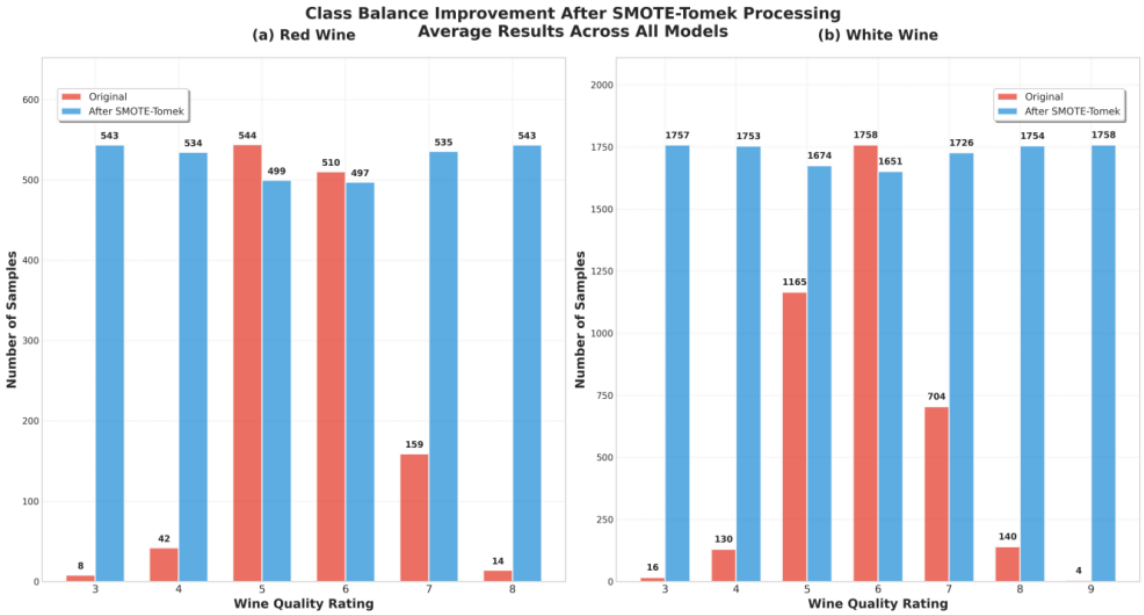


Figure 2: Class Balance Improvement after SMOTE-Tomek Processing.

2% and 0.5%, Now, they contribute roughly 17% and 14% of the training data, and the once dominant categories have also been compressed to similar levels. The resulting nearly uniform class ratio provides balanced evidence for model learning and eliminates the bias caysed by the original skewness.

Class imbalance is tackled both in the data space and the optimization objective. After SMOTE-Tomek resampling, we inversely proportional the loss weight to the class frequency, compelling learners to attach equal importance to each quality grade. Random Forest and Gradient Boosting achieve this through a "balanced" class-weight flag, while XGBoost, LightGBM, and CatBoost obtain an explicit instance weight vector from the same anti-frequency formula. We recalculated the weights of each fold to ensurre they always matched the class distribution of the fold. This dual remedy—balanced data plus cost-sensitive losses—directly translates into performance improvements in balanced perception metrics.

Validation and test segmentation are fully used for untouched references. Each validation fold only accepted the feature scale that fits the paired training fold. No resampling, weighting or synthesis is generated. Therefore, the classification ratio and physicochemical ranges remained unchanged, thereby generating performance estimates that match the deployment conditions. To prevent leakage, each preprocessing parameter was learned within the training segmentation, and all.synthetic samples are restricted within this segmentation. StratifiedGroupKFold further locked the relevant records into a single partition, so enhanced or near-repeated observations do not span training and validation. This strict sequence partitioning, preprocessing, training, evaluation of hidden shortcuts in the original data blocks prevent metric inflation and provide unbiased model selection.

2.2 Overview of Applied Models

We conducted benchmark tests on five gradient tree ensembles, because tree ensembles dominate in structured-data tasks, can handle feature scaling well, and accept class weighting to address residual imbalance issues. Simpler baselines (logistic regression, k-NN, SVM) were reserved only for integrity checks and are not further discussed. Random Forest constructs a T class-probability estimator $\{H_t\}_{t=1}^{T}$ by fitting a copy of the training set from a CART-type tree. Each segmentation considers a random sub-space of $m = \sqrt{d}$ features, forcing the de-correlation between trees and lowering the generalisation error range.

$$E_{RF} \leq E_{tree} \frac{1-\rho}{T} + \rho\, E_{tree} \tag{2}$$

The prediction is obtained through majority voting on the posterior of the of terminal-nodes. Out-of-bag samples provide unbiased error estimate and feature importance based on the Gini-based coefficient.

Gradient boosting approximates the optimal additive model $F(x) = \sum_{t=1}^{T} \eta_t f_t(x)$ that minimises any differentiable loss $L(y, F(x))$. In the iteration, the algorithm computes the negative gradient $g_{it} = -\partial L/\partial F|_{F_{t-1}}$ for each instance i, fits a depth-limited tree f_t to $\{(x_i, g_{it})\}$ and update $F_t = F_{t-1} + \nu\eta_t f_t$ with shrinkage $\nu < 1$ and subsampling $0 < s \leq 10 < s \leq 1$ to control variance.

XGBoost improves gradient boosting by using a loss second-order Taylor expansion of the loss and a ↕1 + ↕2 regularizer on leaf weights:

$$\Omega(f) = \gamma\, k + \frac{1}{2}\lambda \sum_{j} w_j^2 \tag{3}$$

where $\gamma \geq 0$ is an ↕0-style complexity penalty that charges a fixed cost per leaf (k is the number of leaves). A larger γ raises the

gain threshold required for splitting, encouraging shallower, better-generalised trees; typical values range from 0 (no penalty) to 10^{-2} – 10^{1} depending on data complexity. Then., the greedy split selection maximizes the regularized gain:

$$\Delta L \approx \frac{1}{2}\left(\frac{G^2}{H+\lambda} - \frac{G_L^2}{H_L+\lambda} - \frac{G_R^2}{H_R+\lambda}\right) - \gamma \tag{4}$$

Therefore, it is only accepted when the improvement of a candidate split exceeds γ. Block-wise sparsity handling, column subsampling, and histogram caching provide strong overfitting protection while maintaining the per-tree complexity at O(Kdlogn).

LightGBM is enhanced through gradient-based unilateral sampling (GOSS), acceleration retaining all instances with large gradient magnitudes and randomly sampling cases with low gradients to approximate the information gain. Histogram grouping compresses continuous attributes into fixed buckets to reduce memory from O(nd) to O(Bd). The leaf-wise growth strategy with depth constraints can minimize the loss reduction ΔL per split, leading to deeper, narrower trees and faster convergence.

CatBoost employs a symmetrical forgetting tree, whose decision rules are the same on each root-to-leaf path, therby achieving efficient SIMD computations. To suppress target leakage, ordered argumentation builds each tree with incremental permutations, so that each training sample can be predicted by the previously permutation trained model. The classification predictors is encoded with Bayesian-smoothed target statistics:

$$\mathrm{TS}(x) = \frac{\sum_{j<i} 1_{\{x_j=x\}}\, y_j + aP}{\sum_{j<i} 1_{\{x_j=x\}} + a} \tag{5}$$

Here, P is the prior and regularization term, which reduces the prediction offset on small folds. All five models take in features scaled by StandardScaler, balanced by SMOTE-Tomek, and weighted by the inverse class frequency. The hyperparameters are adjusted in the five-fold Stratified Group K-Fold to ensure the accuracy, robustness and runtime comparison under the same data conditions.

2.3 Model Training Process and Evaluation Metrics

2.3.1 Hyperparameter Optimisation. Each classifier was tuned using Optuna 3.5, the Tree-structured Parzen Estimator sampler and median pruner, which discarded any trials with a mid-term weighted F_1 lower than the study median in the same step. The Five-fold groupkfold ensured the layering of quality labels while preventing leakage between batches of wine. Weighted F_1 is the sole optimization objective.

Each model is allocated a fixed trial budget in proportion to the dimension of its search space: 200 trials for Random Forest and XGBoost, 200 for LightGBM, 150 for CatBoost, and 120 for the scikit-learn Gradient Boosting baseline. In each experiment, the gradient booster applied an internal validation delay. If no gain was observed after 10 consecutive iterations, it stops to suppress overfitting and accelerate convergence.

The search space summarized in Appendix 1, has been intentionally expanded. The ree set of Random Forest ranges from 200 to 1,000 estimators, 100 to 600 for Gradient Boosting, while boosting libraries ranged from 200 to 800 trees in XGBoost, 300 to 1 500 in LightGBM, and 100 to 500 in CatBoost. The maximum depth varies by algorithm—up to 30 for Random Forest, 14 for Gradient Boosting, 12 for XGBoost, 20 for LightGBM, and 10 for CatBoost—ensuring that both shallow and deep architectures were examined. The learning rate grid also spanned two orders of magnitude: 5×10^{-3}–4×10^{-1} (log-scale) for Gradient Boosting, 10^{-2}–2×10^{-1} for XGBoost, 5×10^{-3}–5×10^{-1} for LightGBM, and 10^{-2}–3×10^{-1} for CatBoost. Sub-sampling was optional, but it allows for the start/stop of Random Forest, 0.70–1.00 for Gradient Boosting and XGBoost—while LightGBM varied its feature_fraction from 0.40 to 1.00 and XGBoost its column-sampling ratio from 0.60 to 1.00. Uniformly solve the problem of class imbalance by allowing 'balanced' or 'no' class weight settings in each library. Gamma, min_child_weight, and regularization coefficients in XGBoost, num_leaves and GOSS boosting type in LightGBM, or l2_leaf_reg and Bayesian bootstrapping temperature in CatBoost further expanded the exploration envelope.

2.3.2 Two-Stage Feature Selection. We have implemented a two-stage feature selection protocol to identify key predictive features while training the models:

Stage 1 (Screening): Each model was first adjusted on a complete set of 11 physicochemical variables. By leveraging its native importance metrics such as the gain of tree boosters and the average reduction of impurities in the Random Forest, the model generated an important vector, which was normalized to a sum of 1. In red wine, alcohol, volatile acidity, and sulphates accounted for 40% of the total important mass (about 0.405). For white wine, the three main components were transformed into free sulfur dioxide, alcohol, and chloride, which together accounted for 32% of the mass (about 0.319) (Figure 3a).

Stage 2 (Refitting): Based on these rankings, the top five features were selected for each model, applied a minimum important cut-off value of 5% to exclude features whose influence can be ignored. Then, each model was retrained using only the top five features to assess any performance changes. Figure 3 shows three key empirical results. In red wine, all five models retained alcohol, volatile acidity, sulphates, and chlorides, and four of the five models also retain total sulfur dioxide. The white wine models showed an approximately mirror-like pattern, with free sulfur dioxide and fixed acidity replacing the latter two vats (Figure 3b). The heatmap emphasizes that LightGBM is the only learner that maintains the pH value in both wines, while Gradient Boosting systematically reduces the pH value; Random Forest maintained the fixed acidity of white wine alone (Figure 3c).

When the feature space is compressed from 11 to 5, the average absolute change of weighted F_1 0.043 (approximately 6.6% relative to the average score of the entire model at 0.65) (Figure 3d). CatBoost has the greatest impact on white wine, at 0.084. No model saw a performance improvement after pruning, but each drop remained below 9% points, which is acceptable considerring the 55% reduction in dimensions. This result indicates that even with the revised data, a compact set of five easily measurable variables is sufficient for high-fidelity and high-quality predictions, reducing computational load and simplifying interpretation without causing catastrophic loss of accuracy.

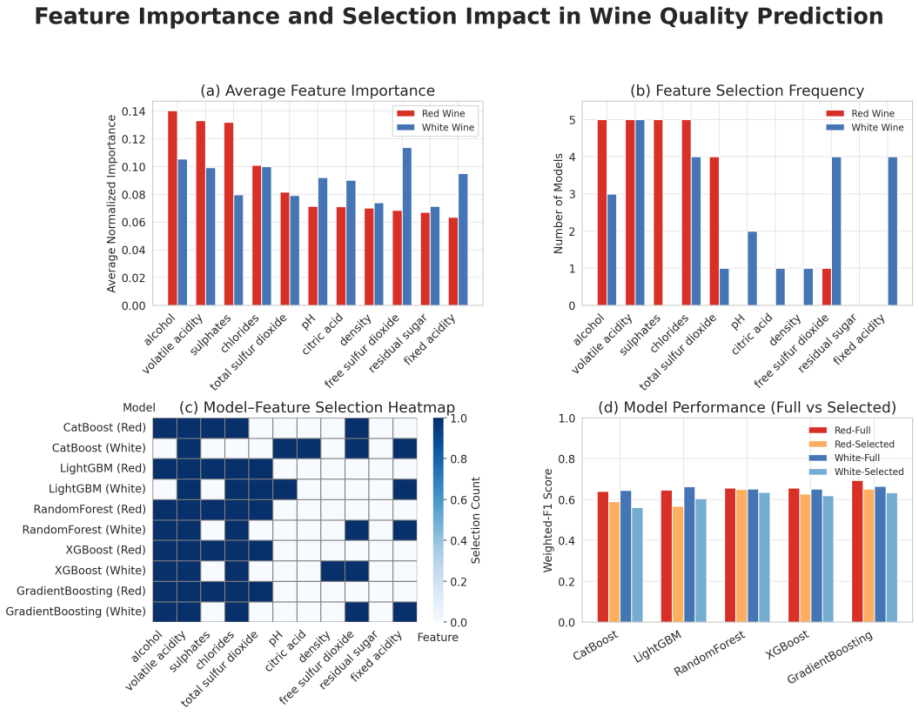


Figure 3: Feature Importance and Selection Impact Across Models in Wine Quality Prediction.

2.3.3 Evaluation Metrics. The performance of the model was primarily quantified by the weighted F_1 score, which is the weight average of the class-frequency of the per-class F_1 values. The weighted F_1 for C classes is defined as: (a) (b)

$$F1_{\text{weighted}} = \frac{1}{N} \sum_{c=1}^{C} n_c \frac{2\, P_c\, R_c}{P_c + R_c} \tag{6}$$

where P_c and R_c are the precision and recall for class C, n_c is the number of instances in class C, and $N = \sum_c n_c$ is the total number of instances. weighted F_1 was chosen as the primary metric to account for the imbalanced class distribution in wine quality labels. In addition, we report macro-averaged F_1 and macro ROC-AUC to evaluate performance across classes independent of their frequencies. The macro F_1 treats all classes equally, highlighting how well the model performs on minority classes, while the macro ROC-AUC provides a threshold-insensitive measure of the overall ranking performance of the classifier across classes. We also include the Matthews correlation coefficient as a single summary statistic of prediction quality; MCC can be interpreted as a correlation coefficient between predicted and true labels, ranging from −1 through 0 (no better than random) to 1 (perfect prediction). Finally, the Brier score is reported to assess calibration of the predicted probabilities, with lower Brier scores indicating more reliable probability estimates.

2.3.4 Computational Considerations. All experiments were executed on a high-performance computing node with an AMD EPYC 9K84 processor (128 logical cores) and 512 GB of RAM, equipped with a single NVIDIA H20 GPU. Model training and hyperparameter search jobs were parallelized on up to eight concurrent worker processes to take full advantage of the available CPU cores and GPU acceleration. For GPU-accelerated training with CatBoost, GPU memory usage was explicitly capped at 70% to prevent memory contention and ensure stable execution. Reproducibility was maintained by fixing the random seed for all training and optimization processes. The software environment consisted of Python 3.10 on Ubuntu 22.04, with core libraries and frameworks including Optuna 3.5 for hyperparameter optimization, scikit-learn 1.4 for cross-validation and metric computation, and XGBoost 2.x, LightGBM 4.x, and CatBoost 1.2 for model implementations.

3 RESULTS

3.1 Model Performance Comparison

On the full eleven-variable input, Gradient Boosting dominates both datasets, attaining weighted F_1 scores of 0.693 ± 0.028 on red wine and 0.664 ± 0.016 on white. Random Forest and XGBoost form a second tier whose weighted F_1 scores differ from the leader by at most three percentage points, while LightGBM falls marginally below this pair and CatBoost trails the group. The auxiliary metrics corroborate this ordering: Gradient Boosting secures the highest Matthews correlation (0.527 red, 0.501 white) and sustains macro AUC above 0.81, whereas CatBoost records the lowest macro F_1 ($\approx$0.40) and the weakest MCC across both datasets. Figure 4 translates these values into four compact line charts for cross-scenario comparison. (The full model metric summary is provided in Table 2).

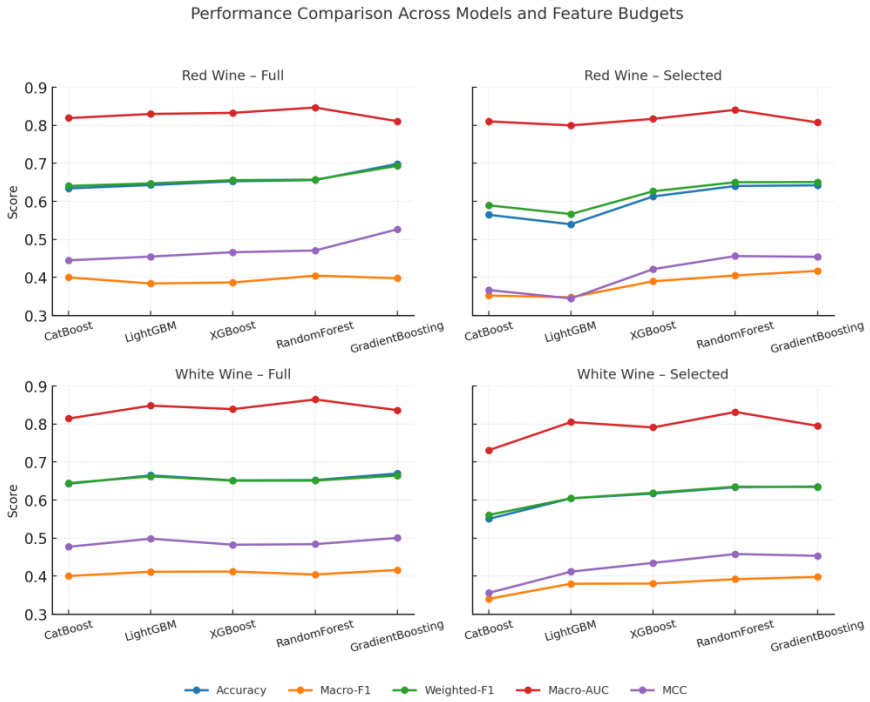


Figure 4: Performance Comparison Across Models and Feature Budgets.

Restricting each model to its five most influential features causes a universal but modest degradation. The median weighted F_1 drop is 2.6 percentage points for red wine and 3.0 for white. Gradient Boosting and Random Forest prove most resilient, retaining weighted F_1 scores of 0.651 on red and 0.635 on white and thus preserving their leading status. XGBoost shows an intermediate decline yet remains competitive. LightGBM and CatBoost are markedly more sensitive: LightGBM loses 8.1 points on red wine, while CatBoost forfeits 8.4 points on white, confirming that both learners rely on the discarded variables for minority-class delineation. The macro F_1 trajectory is consistent with these trends; only Gradient Boosting registers a slight macro F_1 gain after pruning, indicating a favorable precision–recall rebalancing, whereas CatBoost's macro F_1 sinks below 0.36.

Macro AUC values stay above 0.79 in every setting, evidencing stable ranking quality despite feature removal. MCC mirrors weighted F_1 throughout, reinforcing its role as a concise global indicator. Collectively, the results establish Gradient Boosting as the most accurate and robust approach, Random Forest as the most competitive lightweight alternative, and highlight CatBoost's pronounced dependence on the full attribute spectrum.

3.2 Error Analysis of the Poorest-Performing Model

CatBoost achieved the lowest weighted F_1. In the full feature space, the accuracy rates of red and white wine are 0.640 ± 0.027 and 0.645 ± 0.019. The macro F_1 values are only 0.400 ± 0.040 and 0.400 ± 0.013, indicating insuficient recall at the niche level. When the model was retained on the five top-ranked variables, The weighted F_1 dropped by roughly 5 points, highlighting the reliance on weaker features of fine-grained class separation.

Figure 5a shows the normalized confusion matrix of CatBoost on the red wine data. The accuracy rate is concentrated on modal scores of 5 and 6, but 24% of true-5 samples are upgraded to 6 and 22% of true-6 samples are downgraded to 5, indicating that the decision boundary is blurred at the distribution peak. The recall rates of Classes 3 and 8 are less than 5%, and their instance crashes are in a central mode. Therefore, the integrated shallow depth cannot preserve the sparse areas of the feature space. When the model was retrained on its top five variables (Figure 5b), the weighted F_1 decreased from 0.640 to 0.590, and the macroscopic F_1 decreased from 0.400 to 0.352. The correct prediction at level 5 dropped by 13 points, while the incorrect classification spread along the $5 \rightarrow 6 \rightarrow 7$ corridor. Extreme labels have almost completely vanished, confirming that variables such as pH and residual sugar, encode minority clues indispensable for subtle discrimination.

Figure 6a tracked the weighted F_1 returned by CatBoost in each Optuna trial on the complete feature set. In the first 10 tests, the performance rose sharply, exceeding 0.64 in the 20th test, and then fluactuated within a narrow range of ±0.01. A few outliers dropped below 0.55, reflecting the early pruning configuration that combines shallow depth with active Bayesian bagging. Figure 6b shows the equivalent search for five feature subsets. The trajectory converges quickly again, but the upper limit moves downward to ≈0.59, with increased variability, indicating that the reduced attribute space leaves fewer high-quality parameter combinations and amplifies the impact of suboptimal deep learning rates.

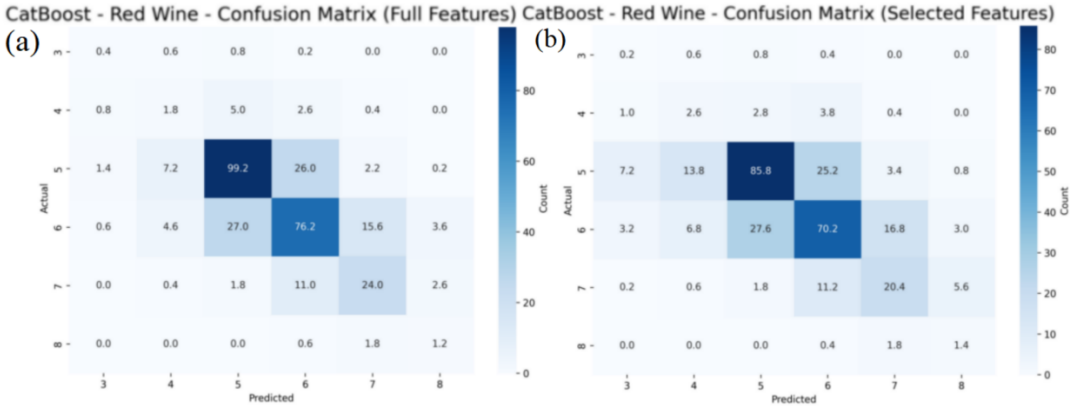


Figure 5: Confusion Matrix of CatBoost on Red Wine.

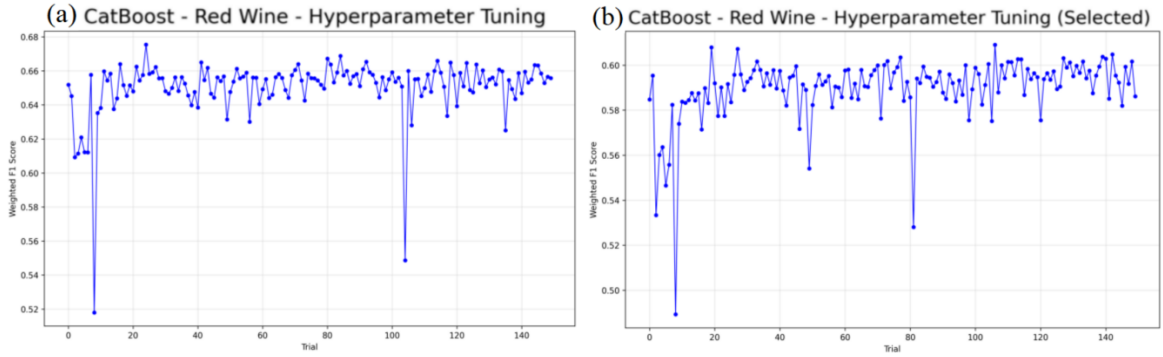


Figure 6: Hyperparameter Tuning Curve of CatBoost on Red Wine Data.

These curves reveal two constraints. First, the search was already saturated before the 150-trial budget was exhausted, indicating that envelopes with a depth of 10 had already limited the model's expressiveness. Second, feature pruning reduces the achievable optimal value by roughly 5 points, indicating that discarded variables provide indispensable supplementary segmentation for the minority classes.

Two remedial measures emerged. Allowing deeper or class-adaptive trees will expand the search configuration, allocating capacity to rare labels without exceeding GPU memory. Restoring two additional variables—pH and residual sugar—can enhance the platform while maintaining a dimension one-third lower than the original input. These adjustments are expected to narrow the performance gap between CatBoost and leading gradient boosting models.

3.3 Practical Considerations

In the real-world wine-quality screening, as long as new vintages, sensor calibrations or process adjustments change the underlying data distribution, these models must be retrained. Therefore, the time required to complete a full hyperparameter search and evaluation cycle directly translates into production downtime, computing cluster rental costs, energy consumption, and carbon footprint. The slower pipes also delay the feedback to the winemaker, hindering the rapid A/B iteration of fermentation parameters. Therefore, training latency is not a peripheral metric but a first-order constraint that must be weighed against prediction accuracy when choosing deployment candidates. Figure 7 shows the highest accuracy of Gradient Boosting, but it takes about 12 hour for every five cycles, which is 10 times slower than its competitors. LightGBM and XGBoost can be completed within 2 to 3 hours using GPUs, CatBoost in about 1 hour, while CPU-based Random Forest converges within 50 minutes. In combination with Table A.2, Random Forest offers the best production trade-offs—the second-highest weighted F_1 (0.657 red, 0.651 white), leading macro AUC, and a minimal pruning loss of 5% at the Gradient Boosting's run. XGBoost is a balanced GPU alternative, while CatBoost's sensitivity to feature reduction limits its value. Therefore, Random Forest is the economic default, and XGBoost is prioritized for fast GPU retraining.

4 CONCLUSION

This study provides the first leak-free, end-to-end benchmark of five ensemble tree learners for multi-class wine-quality prediction on the canonical Vinho Verde datasets. Gradient Boosting sets a performance ceiling but it takes an average of 12 hours for every fivefold optimization, which is an order of magnitude slower than any competitor. Random Forest completes the work of commmercial CPUs within 49 minutes and achieves this accuracy within

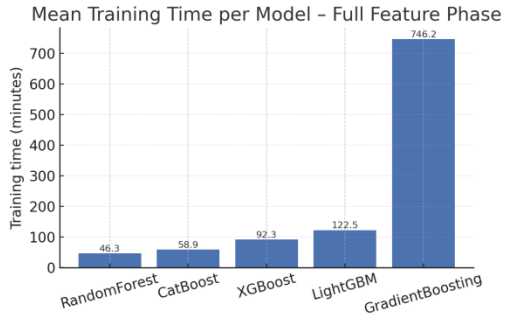


Figure 7: Model Training Time Comparison.

2-3% points, establishing the most cost-effective default value for regular laboratory deployments. Compared with GPU, XGBoost and LightGBM, the marginal performance decline of 2–3 hours for trimming training time makes them attractive to the fast retraining pipelines. However, CatBoost is highly sensitive to feature pruning and does not have a distinct advantage.

Two-stage importance-driven pruning uses only five physicochemical variables to maintain a full model accuracy of ≥93%, thereby achieving lower-cost sensing and faster inference. The combination of SMOTE-Tomek resampled and anti-frequency weighting reduced the class imbalance ratios from 20.7 to 1.04 (red) and 82.9 to.02 (white), generating macroscopic F_1 and MCC gains without increasing the test error. Therefore, we recommend five-feature Random Forest for production (weighted $F_1 \approx 0.65$, macro AUC ≥ 0.86, sub-hour training, interpretable impurity importances); When marginal accuracy gain proves the rationality of GPU usage, XGBoost or LightGBM becomes the preferred choice, while Gradient Boosting is reserved for offline audits where peak accuracy outweighs cost.

Limitations m,itigate these implications: (1) All samples are fromVinho Verde wines in Portugal, and thus unioversality for other regions, varieties or vintages has not been untested; (2) The actual category deviation may deviate from the resampling settings and requires continuous balibration. (3) Modern deep table networks still do not appear in benchmarks.

Future work should verify the integrated baseline under different terroir conditions, assess the robustness under natural imbalances, and explore transformer-based or hybrid architectures when larger datasets emerge. To promote this progress, we have released preprocessing scripts, hyperparameter grids, and feature selection protocols as repeatable baselines for the community.

References

[1] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Wine Quality (data set). UCI Machine Learning Repository.
[2] Budnyak, A. (2020). Wine Rating & Price (data set). Kaggle. (Accessed 27 August 2020)
[3] Budnyak, A. (2020). Wines: EDA and Rating Prediction. Kaggle Notebook (September 3, 2020).
[4] Palmer, J., & Chen, B. (2018). Wineinformatics: Regression on the Grade and Price of Wines through Their Sensory Attributes. Fermentation, 4(4), 92.
[5] Liu, Z. (2023). Comparison of the red wine quality prediction accuracy using 5 machine learning models. Highlights in Science, Engineering and Technology, 11, 36–45.
[6] Patkar, G. S., & Chakkaravarthy, M. (2022). Exploration of Wine Features Using Data Analytics. Proc. INCOFT 2022, 1–4.
[7] Zaza, S., Atemkeng, M., & Hamlomo, S. (2023). Wine Feature Importance and Quality Prediction: A Comparative Study of Machine Learning Algorithms with Unbalanced Data. Proc. SAFER-TEA 2023, 27–33.
[8] Rani, U., Jebamalai, J., & Prabu, C. (2023). Analysis of Multiclass Imbalance Handling in Red Wine Quality Dataset Using Oversampling and Machine Learning Techniques. J. Theor. Appl. Info. Tech., 101(5), 1303–1315.
[9] Dahal, K. R., Dahal, J. N., Banjade, H., & Gaire, S. (2021). Prediction of Wine Quality Using Machine Learning Algorithms. Open Journal of Statistics, 11(2), 278–289.
[10] Ye, K. (2023). Wine quality prediction by several data mining classification models. Highlights in Science, Engineering and Technology.
[11] Kaur, N., Kaur, G., Aruchamy, P., & Chaudhary, N. (2025). An Integrated Approach Based on Fuzzy Logic and Machine Learning Techniques for Reliable Wine Quality Prediction. Procedia Computer Science.
[12] Ma, R., Mao, D., Cao, D., Luo, S., Gupta, S., & Wang, Y. (2024). From vineyard to table: Uncovering wine quality for sales management through machine learning. Journal of Business Research.
[13] Jindal, A., & Singh Gill, K. (2024). From Vineyard Data to Flavour Profiles: Machine Learning Predicts Wine Quality. 2024 12th International Conference on Internet of Everything, Microwave, Embedded, Communication and Networks (IEMECON), 1-6.
[14] Singla, M., Gill, K.S., Chauhan, R., Pokhariya, H.S., & Chanti, Y. (2024). Analysing Red Wine Quality Data Exploratorily using Support Vector Machines and Other Machine Learning Methods. 2024 2nd International Conference on Computer, Communication and Control (IC4), 1-6.
[15] Singla, M., Gill, K.S., Upadhyay, D., & Singh, V. (2024). Exploratory Data Analysis for Red Wine Quality Prediction Using a Decision Tree Approach and Machine Learning Methods. 2024 3rd International Conference for Innovation in Technology (INOCON), 1-5.
[16] Nguyen, Q.D., Le, H.V., Nakano, T., & Tran, T.H. (2024). Wine quality assessment through lightweight deep learning: integrating 1D-CNN and LSTM for analyzing electronic nose VOCs signals. Applied Computing and Informatics.
[17] Díaz, R., & Solares, A. (2025). Wine quality assessment using machine learning. Advances in AI for Industrial Processes (Lecture Notes in Computer Science, Vol. 13823). Springer.

A APPENDIX A

A.1 Hyperparameter grid search space for the three ensemble models.

Table 1: Hyperparameter grid search space for the three ensemble models.

Parameter	RandomForest	GradientBoosting	XGBoost	LightGBM	CatBoost
Trials	200	120	200	200	150
n_estimators	200–1000	100–600	200–800	300–1500	100–500
Depth	10–30	8–14	6–12	10–20	6–10
Learning_rate	-	0.005–0.4 (log)	0.01–0.20(log)	0.005–0.50	0.01–0.30
Subsample	bootstrap {T,F}	0.70–1.00	0.70–1.00	-	-
Min_samples_leaf	1–15	1–8	-	5–200	-
Class_weight	balanced, None	balanced, None	balanced, None	balanced, None	balanced, None
Min_samples_split	2–20	2–15	-	-	-
Max_features	$\sqrt{\log_2}$, 0.3–1.0	$\sqrt{\log_2}$, None	-	-	-
Feature_fraction	-	-	0.60–1.00	0.40–1.00	-
Else parameters	Criterion gini	validation_fraction 0.10–0.20	gamma 0–10	boosting_type goss;	l2_leaf_reg 1–10;
	Criterion entropy	-	min_child_weight 1–15;	num_leaves 100–500;	bootstrap_type Bayesian
	Criterion log_loss	-	reg_alpha 0–2;	lambda_l1/l2 0–10;	bagging_temperature 0.1–0.9
	-	-	reg_lambda 1–15;	extra_trees {T,F}	-

A.2 Overall Models Metric Summary.

Table 2: Overall Models Metric Summary.

Model	Dataset	Phase	Accuracy	Macro-F_1	Weighted-F_1	Macro AUC	MCC
XGBoost	red	full	0.6529	0.3868	0.6558	0.8328	0.4663
XGBoost	red	selected	0.6129	0.3898	0.6265	0.817	0.4218
XGBoost	white	full	0.6519	0.412	0.6511	0.8392	0.4827
XGBoost	white	selected	0.6172	0.3806	0.6192	0.7911	0.4349
LightGBM	red	full	0.6429	0.384	0.6473	0.8299	0.4548
LightGBM	red	selected	0.5397	0.3477	0.5666	0.7998	0.3448
LightGBM	white	full	0.665	0.4116	0.6625	0.8485	0.4985
LightGBM	white	selected	0.6047	0.3801	0.6047	0.8052	0.4118
RandomForest	red	full	0.656	0.4046	0.6571	0.8468	0.4707
RandomForest	red	selected	0.6404	0.4052	0.6503	0.8407	0.4561
RandomForest	white	full	0.6527	0.4044	0.6513	0.8647	0.4841
RandomForest	white	selected	0.6339	0.3921	0.6352	0.8318	0.4582
GradientBoosting	red	full	0.6986	0.3979	0.6933	0.8108	0.5265
GradientBoosting	red	selected	0.6423	0.4171	0.6508	0.8075	0.4542
GradientBoosting	white	full	0.6699	0.4162	0.6644	0.8365	0.5006
GradientBoosting	white	selected	0.6356	0.3979	0.6345	0.7953	0.4535
CatBoost	red	full	0.6341	0.4	0.6406	0.8191	0.445
CatBoost	white	full	0.6429	0.4004	0.6448	0.8145	0.4773
CatBoost	red	selected	0.5647	0.3522	0.5895	0.8103	0.3667
CatBoost	white	selected	0.5506	0.3404	0.5609	0.7312	0.3559